

Review coupling và cohesion cùng với nguyên tắc thiết kế SOLID Tuần 1

Khi đánh giá thiết kế của bạn về mặt **coupling** và **cohesion**, cũng như xem xét liệu nó có tuân thủ nguyên tắc **SOLID** hay không, chúng ta sẽ đi qua các yếu tố sau:

I. Coupling

Coupling đo lường mức độ phụ thuộc giữa các lớp trong hệ thống. Mục tiêu là làm sao để giảm sự phụ thuộc giữa các lớp để hệ thống dễ bảo trì và mở rộng hơn. Các loại coupling mà bạn yêu cầu sẽ được đánh giá như sau:

1. Content Coupling:

- **Đánh giá:** Các lớp như `ProductCart``, `OrderProduct``, `TransactionInfo`` có thể được coi là có coupling nội dung khi chúng cần phải truy cập các thuộc tính hoặc thực hiện logic của các lớp khác để hoạt động (ví dụ như việc xử lý sản phẩm trong giỏ hàng, hay giao dịch liên quan đến hóa đơn).
- **Nhận xét:** Mức độ coupling này có thể được giảm bằng cách di chuyển logic xử lý ra khỏi các entity, thay vào đó sử dụng các dịch vụ (services) để xử lý các nghiệp vụ này.

2. Common Coupling:

- **Đánh giá:** Lớp `BaseEntity`` được chia sẻ giữa các lớp khác (ví dụ: `Product``, `User``, `TransactionInfo``), cung cấp các thông tin chung như thời gian tạo và cập nhật.
- **Nhận xét:** Đây là một loại coupling chung hợp lý, vì nó giảm thiểu việc lặp lại mã (DRY). Tuy nhiên, việc các lớp khác kế thừa `BaseEntity`` có thể dẫn đến sự phụ thuộc lớn vào các thuộc tính này.

3. Control Coupling:

- **Đánh giá:** Hệ thống có sự điều khiển giữa các lớp thông qua các hành động như việc chuyển đổi trạng thái `isPaid`` trong `Order`` hay `isRush`` trong `Order``. Các lớp có thể yêu cầu cập nhật trạng thái của lớp khác (ví dụ, `OrderProduct`` liên kết với `Order`` và `Product``).
- **Nhận xét:** Mặc dù sự kiểm soát này là cần thiết cho việc quản lý trạng thái, nhưng nó cần phải được hạn chế để tránh việc một lớp phụ thuộc quá nhiều vào các logic xử lý của lớp khác.

4. Stamp Coupling:

- **Đánh giá:** Các lớp như `Order``, `Invoice``, `ProductCart`` chia sẻ nhiều thuộc tính dữ liệu với nhau (ví dụ: `Order`` chứa nhiều `OrderProduct``, `ProductCart`` chứa nhiều `Product``).
- **Nhận xét:** Đây là một dạng coupling khá cao, khi một lớp yêu cầu nhiều dữ liệu từ lớp khác. Tuy nhiên, việc này có thể giảm bằng cách sử dụng các đối tượng chuyển tiếp (DTO) hoặc phân tách các trường dữ liệu cần thiết.

5. Data Coupling:

- Đánh giá: Các lớp `Product`, `UserCart`, `Order` chỉ chia sẻ các dữ liệu cơ bản như ID, trạng thái, số lượng, giá trị.

- Nhận xét: Đây là một dạng coupling lý tưởng vì chỉ có dữ liệu được truyền giữa các lớp, không có sự phụ thuộc vào hành vi hay logic của lớp khác.

II. Cohesion

Cohesion đo lường mức độ liên kết giữa các chức năng trong một lớp. Lớp có cohesion cao là lớp chỉ thực hiện một chức năng duy nhất và các phương thức trong lớp đều phục vụ cho mục đích đó. Đánh giá cohesion trong thiết kế của bạn như sau:

1. Functional Cohesion:

- Đánh giá: Các lớp như `Product`, `User`, `Order`, và `Invoice` đều thực hiện chức năng rõ ràng, như lưu trữ thông tin sản phẩm, thông tin người dùng, hay các giao dịch của đơn hàng. Các lớp này chỉ chứa các thuộc tính và phương thức liên quan đến công việc cụ thể của chúng.

- Nhận xét: Đây là cohesion cao, vì mỗi lớp có một trách nhiệm riêng biệt rõ ràng.

2. Procedural Cohesion:

- Đánh giá: Các lớp không rõ ràng thể hiện tính procedural cohesion. Hầu hết các lớp chỉ lưu trữ dữ liệu và không chứa quá nhiều logic xử lý.

- Nhận xét: Hệ thống có thể cần phải cải thiện ở mức độ này bằng cách tách các xử lý phức tạp ra khỏi các entity và đưa vào các lớp dịch vụ.

3. Sequential Cohesion:

- Đánh giá: Các lớp như `Order` và `ProductCart` có thể có sự liên kết tuần tự, ví dụ khi một đơn hàng được tạo ra, sản phẩm sẽ được thêm vào giỏ hàng.

- Nhận xét: Đây là một dạng cohesion hợp lý khi các lớp làm việc cùng nhau trong một chuỗi hành động.

4. Logical Cohesion:

- Đánh giá: Các lớp trong hệ thống có tính chất logical cohesion, ví dụ như `ProductCategory` trong `Product` cho phép phân loại các sản phẩm theo các danh mục `CD`, `DVD`, `BOOK`.

- Nhận xét: Cohesion này là hợp lý vì các lớp hoạt động độc lập và có thể phân loại các đối tượng trong hệ thống.

5. Incidental Cohesion:

- Đánh giá: Các lớp như `TransactionInfo` và `Invoice` có thể có một số chức năng ngoài việc lưu trữ dữ liệu, ví dụ như xác nhận giao dịch hay cập nhật trạng thái của đơn hàng.

- Nhận xét: Mặc dù có thể tồn tại sự phụ thuộc nhất định, nhưng chúng cần cải thiện để chỉ tập trung vào một mục đích cụ thể.

6. Communicational Cohesion:

- Đánh giá: Các lớp như `Order`, `Invoice`, và `TransactionInfo` chia sẻ thông tin giao dịch và đơn hàng, điều này giúp tăng cường giao tiếp giữa chúng.

- Nhận xét: Tính cohesion này là hợp lý và phù hợp với mục đích của các lớp này.

III. Nguyên tắc thiết kế SOLID

1. S – Single Responsibility Principle (SRP):

- Đánh giá: Các lớp như `Product`, `User`, `Order`, `Invoice` đều thực hiện một nhiệm vụ duy nhất, ví dụ như `Product` chỉ lưu trữ thông tin về sản phẩm, `Order` chỉ quản lý đơn hàng. Điều này giúp đảm bảo rằng mỗi lớp có một trách nhiệm rõ ràng.

- Nhận xét: SRP được tuân thủ tốt trong thiết kế của bạn.

2. O – Open/Closed Principle (OCP):

- Đánh giá: Các lớp như `Product`, `User`, và `Order` được thiết kế để mở rộng nhưng không thay đổi, vì bạn có thể thêm các thuộc tính mới hoặc thay đổi hành vi mà không làm ảnh hưởng đến cấu trúc hiện tại.

- Nhận xét: Được tuân thủ khá tốt, nhưng có thể cải thiện việc mở rộng các lớp dịch vụ thay vì thay đổi entity trực tiếp.

3. L – Liskov Substitution Principle (LSP):

- Đánh giá: Các lớp con như `ProductCart` hay `UserCart` có thể thay thế các lớp cha mà không gặp vấn đề gì, vì chúng không thay đổi hành vi của lớp cha `BaseEntity`.

- Nhận xét: LSP được thực hiện đúng đắn trong hệ thống.

4. I – Interface Segregation Principle (ISP):

- Đánh giá: Các lớp của bạn không quá phụ thuộc vào các giao diện lớn và không cần triển khai nhiều phương thức không cần thiết. Hệ thống có thể chia nhỏ các giao diện dịch vụ để tuân thủ nguyên tắc này.

- Nhận xét: ISP có thể được cải thiện bằng cách chia các giao diện dịch vụ ra thành các phần nhỏ hơn.

5. D – Dependency Inversion Principle (DIP):

- Đánh giá: Hệ thống hiện tại có sự phụ thuộc trực tiếp giữa các lớp entity và lớp dịch vụ. Để tuân thủ DIP, các lớp entity không nên phụ thuộc trực tiếp vào các lớp dịch vụ mà thay vào đó, các lớp dịch vụ nên phụ thuộc vào các giao diện.

- Nhận xét: DIP chưa được thực hiện triệt để, vì hệ thống vẫn có sự phụ thuộc chặt chẽ giữa các lớp. Cần cải tiến thiết kế để giảm phụ thuộc này.

IV. Tóm lại:

1. Coupling: Hệ thống của bạn có sự phụ thuộc hợp lý, nhưng vẫn có thể giảm coupling giữa các lớp bằng cách sử dụng các dịch vụ (services) để xử lý logic thay vì trực tiếp trong các entity.
2. Cohesion: Các lớp của bạn có tính cohesion khá cao, nhưng có thể cải thiện hơn nữa bằng cách đảm bảo mỗi lớp chỉ làm một việc duy nhất và tránh chứa quá nhiều logic.
3. SOLID: Hệ thống tuân thủ khá tốt nguyên tắc SRP, OCP, và LSP. Tuy nhiên, cần cải thiện ISP và DIP để đạt được mức độ thiết kế tối ưu hơn.